GREENHERB API

QA and Testing Report

Diego Alonso Laguillo and David Gonzalez Ferenandez

1. Introduction

This report covers te work developed regarding the REST API of Greenherb´s platform, which is oriented to develop an automatic intelligent managements of greenherbs.The objective was to design,implement and execute a set of tests to validate the functionality of the API, applying different testing approaches and compiling the results.

The project statement says that the platform must be evaluated using different testing levels: unit tests, integration tests, and system tests, as well as by applying black-box and white-box techniques like equivalence partitioning, boundary value analysis, and MC/DC. It also requires documenting the process through a traceability matrix, execution evidence, coverage metrics, and identified defects.

2. System developed

The system tested is a REST API done in Java 17 and Spring Boot.

API includes this main modules:

Module	Endpoint	Description
Auth	/auth/login	User login and JWT token generation
Plans	/plans	Creation and retrieval of crop plans
Measurements	/measurements	Temperature and humidity monitoring

Alerts	/alerts	Alert retrieval, resolution, and dismissal
Swagger	/swagger-ui/index.html	Visual API documentation

The system uses JWT authentication, an in-memory H2 database, Swagger/OpenAPI documentation, and a layered architecture composed of controllers, services, repositories, and models.

3. Used technologies

Tools	Usage
Java 17	Backend language
Spring Boot	API framework
Spring Security	Endpoint security and authentication
JWT	Authentication using tokens
H2 Database	Use of an in-memory database for development and testing purposes
Maven	Dependency management and testing execution
JUnit 5	Unit testing and integration testing
Mockito	Mock creation in unit tests
MockMvc	Simulating HTTP requests in integration tests
Postman	Manual and automated system testing

Newman	Automated Postman collection execution
JaCoCo	Code coverage
Swagger	Endpoint documentation and manual testing
GitHub	Version control and project repository management

4. Sprint-based work organization

Sprint 1

Sprint 1 mainly required:

- endpoint creation
- unit test creation for authentication
- report update with the traceability matrix

During this sprint, the backend foundation was implemented and made operational:

- Spring Boot project creation
- package structure configuration
- /auth/login endpoint implementation
- JWT authentication working correctly
- administrator user automatically loaded
- Swagger fully operational
- endpoints protected using token-based authentication
- initial authentication tests implemented

The login process is performed using:

```
{

  "username": "admin",

  "password": "1234"

}
```

The system returns a JWT token that is subsequently used to access protected endpoints.

Sprint 2

Sprint 2 required:

- unit tests for aromatic herb catalog import
- unit tests for cultivation plan creation
- report and traceability matrix updates

In our case, the implemented backend focused on the following modules:

- authentication
- cultivation plans
- environmental measurements
- alerts.

The tests were applied to the functionalities that were actually implemented. Unit tests were carried out for PlanService, MeasurementService, AlertService, and AuthService, in addition to integration and system tests for the existing endpoints.

5. Project structure

General structure of the project

```
greenherb-api
│
├── src
│   ├── main
│   │   └── java
│   │       └── com.greenherb.greenherb_api
│   │           ├── config
│   │           ├── controller
│   │           ├── dto
│   │           ├── model
│   │           ├── repository
│   │           ├── security
│   │           └── service
│   │
│   └── test
│       └── java
│           └── com.greenherb.greenherb_api
│               ├── integration
│               └── service
│
```

```
│
├── docs
│   ├── evidence
│   ├── traceability-matrix.md
│   ├── equivalence-partitioning.md
│   ├── boundary-value-analysis.md
│   ├── mcdc.md
│   └── defects.md
│
├── postman
│   └── GREENHERB-System-Tests.postman_collection.json
│
└── pom.xml
```

6. Implemented Endpoints

6.1. Authentication

Method	Endpoint	Description
POST	/auth/login	Authenticates the user and returns a JWT

This endpoint allows users to log in using the administrator account automatically created when the application starts.

6.2. Cultivation Plans

Method	Endpoint	Description
GET	/plans	Returns all plans
POST	/plans	Creates a new plan
GET	/plans/{id}	Retrieves a plan by ID
PUT	/plans/{id}	Updates a plan
DELETE	/plans/{id}	Deletes a plan

The POST /plans endpoint is protected and requires administrator privileges.

6.3. Measurements

Method	Endpoint	Description
POST	/measurements	Records an environmental measurement

When a measurement is recorded, the system checks the active cultivation plan and generates alerts if the conditions are outside the established limits.

6.4. Alerts

Method	Endpoint	Description
GET	/alerts	Returns all alerts
PATCH	/alerts/{id}/resolve	Resolves an alert
PATCH	/alerts/{id}/ignore	Ignores an alert with justification

The system allows alerts to be resolved or ignored. However, when an alert is ignored, a justification is required.

7. Unit Tests

The unit tests were implemented using JUnit 5 and Mockito. Their purpose is to verify the internal logic of the services in isolation, without starting the server or depending on the real database.

Location:

src/test/java/com/greenherb/greenherb_api/service/

Tests were created for the following classes:

Test Class	Tested Service
AuthServiceTest	AuthService
PlanServiceTest	PlanService
MeasurementServiceTest	MeasurementService
AlertServiceTest	AlertService

7.1. AuthServiceTest

The following authentication behaviors were validated:

- login with correct credentials
- non existing user
- incorrect password
- correct JWT token generation

These tests verify that the authentication service searches for the user, validates the password using PasswordEncoder, and generates a JWT through JwtService

7.2. PlanServiceTest

The following plan creation logic was validated:

- creation of a valid plan
- rejection of a plan where the minimum temperature is greater than or equal to the maximum temperature

- rejection of a plan where the minimum humidity is greater than or equal to the maximum humidity

Example of a tested rule:

```
if (plan.getMinTemp() >= plan.getMaxTemp()) {

   throw new RuntimeException("minTemp must be lower than maxTemp");

}
```

This demonstrates that the system does not accept inconsistent ranges

7.3. MeasurementServiceTest

The system behavior when registering measurements was validated through the following cases:

- measurement within acceptable limits
- generation of a critical alert
- generation of a warning alert
- error when no active plan exists

The main tested logic was:

```
if (highTemperature && lowHumidity) {
   // CRITICAL alert
}

if (highHumidity || lowTemperature) {
   // WARNING alert
}
```

These tests verify that the system correctly detects abnormal environmental conditions and generates the appropriate alert type

7.4. AlertServiceTest

The following alert operations were validated:

- creation of an active alert
- resolution of an existing alert
- error when resolving a non-existing alert
- ignoring an alert with justification,
- error when ignoring an alert without justification.

This covers the business rule that requires a justification when an alert is ignored.

8. Integration tests

The integration tests were implemented using:

- @SpringBootTest,
- @AutoConfigureMockMvc,
- MockMvc,
- H2 database.

Location:

src/test/java/com/greenherb/greenherb_api/integration/

The following integration test classes were created:

Test Class	Tested Endpoints
AuthControllerIntegrationTest	/auth/login
PlanControllerIntegrationTest	/plans
MeasurementControllerIntegrationTest	/measurements
AlertControllerIntegrationTest	/alerts

8.1. AuthControllerIntegrationTest

The following two scenarios were verified:

Test	Expected Result
Login with correct credentials	HTTP 200
Login with incorrect password	HTTP 400

This validates that the login endpoint works correctly and properly rejects invalid credentials.

8.2. PlanControllerIntegrationTest

The following cases were tested:

Test	Expected Result
Create valid plan	HTTP 200
Create plan with invalid temperature range	HTTP 400
Retrieve plans	HTTP 200

In these tests, a real login was performed, a JWT token was obtained, and it was used in protected requests through the following header:

Authorization: Bearer TOKEN

8.3. MeasurementControllerIntegrationTest

The following measurements with different values were tested:

Case	Expected Result
Temperature 23 and humidity 60	No alert generated
Temperature 30 and humidity 30	Generates a CRITICAL alert
Temperature 15 and humidity 60	Generates a WARNING alert

These tests validate the integration between:

- measurement controller
- measurement service
- measurement repository
- plan repository
- alert repository

8.4. AlertControllerIntegrationTest

The main alert operations were tested:

Case	Expected Result
Retrieve alerts	HTTP 200
Resolve alert	RESOLVED status
Ignore alert with justification	IGNORED status
Ignore alert without justification	HTTP 400

This validates that the alert module responds correctly to both valid and invalid operations.

9. System Tests with Postman

The system tests were carried out using Postman, simulating a complete user workflow against the real API.

The tested workflow was:

1. Administrator user login
2. JWT token retrieval
3. Creation of an active cultivation plan

4. Retrieval of plans
5. Registration of a critical measurement
6. Retrieval of alerts
7. Resolution of an alert

Exported collection:

postman/GREENHERB-System-Tests.postman_collection.json

The complete Postman workflow demonstrates that the system works end-to-end, not only through isolated components. This matches what the project statement defines as system testing, where the entire application is evaluated through end-to-end API workflows.

10. JWT Testing

Integration Testing

The tests perform a real login against:

POST /auth/login

After that, they extract the token and use it in subsequent requests:

Authorization: Bearer TOKEN

Postman

The Login request automatically stores the token in an environment variable:

let json = pm.response.json();

pm.environment.set("token", json.token);

Then, the remaining requests use:

{{token}}

This demonstrates that the protected endpoints work correctly with JWT authentication.

11. JaCoCo coverage

JaCoCo was configured in the pom.xml file to generate coverage reports.

The report can be found at:

target/site/jacoco/index.html

Results obtained:

Metrics	Result
Instruction Coverage	82%
Branch Coverage	70%

These results were saved as evidence through screenshots in:

docs/evidence/jacoco-coverage.png

The assignment requires reporting coverage metrics for instructions, branches/decisions, and multiple conditions

12. Equivalence partitioning

Equivalence partitioning was applied to classify valid and invalid inputs. The assignment states that test cases must not be arbitrary, but instead derived using formal techniques.

File:
docs/equivalence-partitioning.md

It was applied to:

Area	Valid Class	Invalid Class
Temperature	Within limits	Below or above limits

Humidity	Within limits	Below or above limits
Plan	Consistent ranges	Inconsistent ranges
Justification	Valid text	Empty or null

Example

Input	Class	Expected Result
Temperature 23	Valid	Accepted
Temperature 30	High invalid	Critical alert
Humidity 30	Low invalid	Critical alert
Empty justification	Invalid	Rejected

13. Boundary value analysis

Boundary value analysis was applied to numerical parameters such as temperature and humidity. The assignment requires testing values such as minimum-1, minimum, nominal, maximum, and maximum+1.

File:
docs/boundary-value-analysis.md

Values used:

Parameter	min-1	min	nominal	max	max+1
Temperature	17	18	23	28	29
Humidity	39	40	60	80	81

14. MC/DC y multiple condition coverage

Multiple condition coverage and MC/DC were documented in:

docs/mcdc.md

The assignment requires applying white-box testing techniques on compound decisions, including MC/DC and combined condition coverage.

14.1. Critical alert condition

Logic used:

if (highTemperature && lowHumidity)

High Temperature	Low Humidity	Resultado
false	false	No alert
true	false	No alert
false	true	No alert
true	true	CRITICAL alert

14.2. Alert warning condition

Logic used:

if (highHumidity || lowTemperature)

High Humidity	Low Temperature	Resultado
false	false	No warning
true	false	WARNING
false	true	WARNING
true	true	WARNING

.

14.3. Plan validation

Logic used:

if (minTemp >= maxTemp)

Condición	Resultado
false	Plan aceptado
true	Plan rechazado

14.4. Ignore alert validation

Logic used:

if (justification == null || justification.isBlank())

Null	Blank	Resultado
false	false	Aceptado
true	false	Rechazado
false	true	Rechazado

15.Traceability Matrix

The traceability matrix was created in:

docs/traceability-matrix.md

The assignment requires that each test case be related to its requirement, endpoint, test level, applied technique, expected result, and preconditions.

Example of traceability:

ID	Requirement	Endpoint	Level	Technique	Result
AUTH-01	Valid login	POST /auth/login	Integration	Equivalence Partitioning	PASS
PLAN-01	Create valid plan	POST /plans	Integration	Equivalence Partitioning	PASS
MEAS-02	Generate critical alert	POST /measurements	Integration	MC/DC	PASS
ALERT-04	Ignore alert without justification	PATCH /alerts/{id}/ignore	Integration	Boundary Value Analysis	PASS

16. Detected defects

During testing, several defects or problematic behaviors were identified. They were documented in:

docs/defects.md

The assignment states that the detected defects must be recorded in the report with their severity, reproduction steps, and status

16.1. DEFECT-01:

Description:
If two active plans are created and then a measurement is recorded, the system throws an error because it expects to find only one active plan.

Observed error:
Query did not return a unique result

Severity:
High

Proposed solution:
Prevent the creation of more than one active plan,
or modify the logic to correctly select a single active plan.

16.2. DEFECT-02

Description:
If a measurement is recorded without an active plan existing, the system returns an error.

Observed error:
No active plan found

Severity:
Medium

Proposed solution:
Return a clearer error message,
validate beforehand that an active plan exists before allowing measurements.

16.3. DEFECT-03

Description:

The system correctly rejects the attempt to ignore an alert without justification.

Observed result:

- HTTP 400 Bad Request
- Severity:
- Low

This behavior confirms an important business rule: an alert cannot be ignored without explaining the reason.

17. Evidence generated

Evidence	Location
Successful Postman execution capture	docs/evidence/postman-run-success.png
JaCoCo coverage capture	docs/evidence/jacoco-coverage.png
Postman collection exported	postman/GREENHERB-System-Tests.postman_collection.json
JaCoCo HTML report	target/site/jacoco/index.html

Unit and integration tests	src/test/java/...

18. Final Results:

Element	Status
Functional backend	Completed
Swagger working	Completed
JWT working	Completed
Unit tests	Completed
Integration tests	Completed
System tests	Completed
Postman collection	Completed
JaCoCo coverage	Completed
Equivalence Partitioning	Completed
Boundary Value Analysis	Completed
MC/DC	Completed
Traceability Matrix	Completed
Defect Report	Completed
Evidence documentation	Completed

19. Limitations

Although the full statement mentions other resources such as /herbs, /batches, /tasks, /automation, /reports, and /audit, the implemented version of the project focuses on:

authentication, plans, measurements, and alerts.

Therefore, testing has been applied to the modules that have actually been implemented. Undeveloped endpoints are outside the current scope of the project.

This limitation should be mentioned in the final report to clarify that requirements have not been ignored, but that the actual scope of the developed backend is smaller.

.

20. Conclusion

The GREENHERB API project has a fully implemented functional base and a robust test suite. The main levels of testing have been covered: unit testing, integration testing and system testing.

In addition, the formal techniques required by the brief have been applied: equivalence partitioning, boundary value analysis, MC/DC, traceability matrix, and defect reporting.

Obtained by Jacoco:

Metrics	Results
Instruction Coverage	82%
Branch Coverage	70%

Overall, the work demonstrates a comprehensive, documented testing strategy aligned with the course objectives. The API has been validated both internally and at the level of real user endpoints and flows, including JWT authentication, plan creation, measurement logging, alert generation and alert resolution.